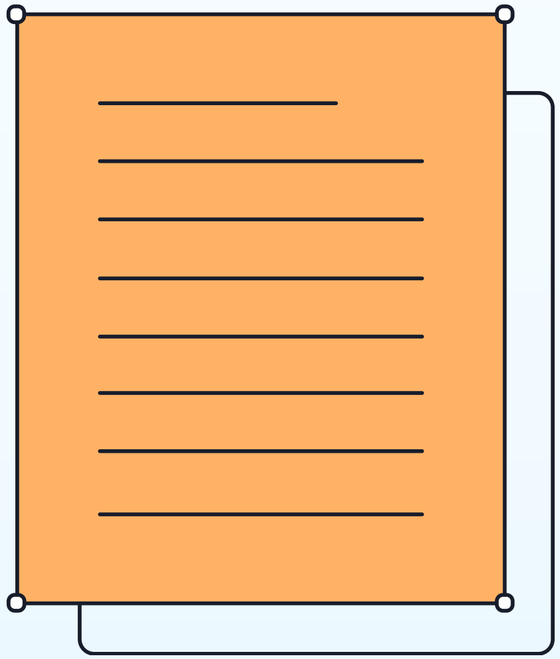
COMPARATIVE STUDY OF MACHINE LEARNING VS DEEP LEARNING MODELS IN FINANCIAL RETURN PREDICTION

Presented by :
Aznak Meryam
El Bouhali Nouhaila



Overview

- Problematic
- Motivation
- Dataset
- Feature Engineering
- Data Split
- ML Models
- DL Models
- Experimental Setup
- Evaluation Metrics
- Results
- Key Findings

PROBLEM STATEMENT
&
MOTIVATION

Problem Statement

Can a machine learn from past stock data to predict tomorrow's return?

Which model predicts financial returns most accurately?

THE CHALLENGE

- Financial markets are difficult to predict.
- Traditional statistical models assume linear relationships.
- Markets often show complex nonlinear patterns.
- Machine learning may capture these patterns better.

Motivation

Our Approach

Directly compare 4 models ; from simple to complex on the same data and same evaluation setup for a fair, reproducible result.

Key Gap

Most studies test one asset or skip real engineering constraints. We tested 4 assets in a scalable PySpark pipeline.

DATASET

Dataset

Source: Yahoo Finance via yfinance library | Train: 2018–2022 Test: 2023–2025

^GSPC	AAPL	MSFT	GOOGL
S&P 500 Index	**Apple Inc.**	**Microsoft**	**Alphabet**
Diversified US market benchmark	High-cap tech, high liquidity	Enterprise tech & cloud leader	Advertising & search giant
8,040	**5**	**20**	**3-day**
Asset-Day Rows	Raw Columns (OHLCV)	Engineered Features	Max Forward-Fill for Gaps

4
Assets

7 Years
2018 – 2025

5
Raw Columns

Dataset

For each day and each asset, we have:

Date	Close	High	Low	Open	Volume	Asset_ID	Asset_ID_encoded
1/2/2018	40.3418846	40.3512543	39.6393091	39.8500839	102223600	AAPL	0
1/3/2018	40.3348656	40.8781924	40.2716363	40.4051238	118071600	AAPL	0
1/4/2018	40.5222092	40.6252541	40.2997277	40.4074539	89738400	AAPL	0
1/5/2018	40.9835815	41.0702314	40.5269081	40.6182428	94640000	AAPL	0
1/8/2018	40.831356	41.1264365	40.7329923	40.831356	82271200	AAPL	0
1/9/2018	40.826664	40.997623	40.6112079	40.8781865	86336000	AAPL	0
1/10/2018	40.8173065	40.8196507	40.5152005	40.5526721	95839600	AAPL	0
1/11/2018	41.0491447	41.0983266	40.8641348	40.8875518	74670800	AAPL	0
1/12/2018	41.4730339	41.5362667	41.1357971	41.2599185	101672400	AAPL	0
1/16/2018	41.2622643	42.0116775	41.2505539	41.6627303	118263600	AAPL	0
1/17/2018	41.9437637	41.9788911	40.9999708	41.2528949	137547200	AAPL	0
1/18/2018	41.9812279	42.1779517	41.7446954	42.0069891	124773600	AAPL	0
1/19/2018	41.7938766	42.0561704	41.5479745	41.829004	129700400	AAPL	0
1/22/2018	41.4519615	41.6346309	41.3582862	41.5222198	108434400	AAPL	0
1/23/2018	41.4613266	42.0233892	41.4098077	41.5222188	130756400	AAPL	0

Open : price when the market opened that morning

Close : price when the market closed that afternoon

High : highest price reached during the day

Low : lowest price reached during the day

Volume : how many shares were traded that day

Target Variable : Prediction Target

We predict <u>next-day return.</u>

Formula:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Pt = price at time t

Pt−1 = price at previous day

Why returns?

- More stable than prices
- Common in financial modeling

FEATURE ENGINEERING

Feature Engineering

Raw prices alone are not very informative, so we created features describing market behavior. These include lagged returns, volatility indicators, and intraday price signals.

Return Features

- daily_return = ln(Pt / Pt-1)
- Return_lag1 to lag5
- Roll_mean5
- Volatility

Price Features

- Open–Close gap
- High–Low range
- Momentum

Volume Features

- Lagged volume
- Volume ratio

Data Split

We split the dataset chronologically so that the model learns from past data and is evaluated on future data to simulate real-world forecasting.

TRAIN — 2018 to 2022	TEST 2023–2025

- **Prevents data leakage**
- **Evaluates future prediction ability**

Comparison of the architecture of ML and DL

Linear Regression

- Simple statistical model
- Input features combined through a linear equation
- Architecture: Input features → Linear weighted combination → Output prediction.
- No hidden layers or complex structure
- Fast to train but limited to linear relationships

Random Forest

- Ensemble architecture of many decision trees
- Each tree trained on a random subset of data and features
- Final prediction = average of all tree outputs
- Captures non-linear relationships
- Architecture: Input data → Multiple decision trees → Aggregated prediction (average).

ANN

- Multi-layer neural architecture
- Architecture: Input layer → Hidden layers (neurons + activation functions) → Output layer.
- Each neuron applies weights, bias, and activation functions
- Learns complex nonlinear patterns

LSTM

- Specialized Recurrent Neural Network (RNN) architecture
- Architecture: Input sequence → LSTM cells (memory gates: input, forget, output) → Prediction.
- Designed to learn temporal dependencies in time-series data
- Suitable for financial market sequences

Key Difference

Machine Learning : simpler structures, faster training, limited temporal modeling.

Deep Learning : deeper architectures, better at capturing complex and sequential financial patterns, but require more data and computation.

CLASSICAL ML : Linear Regression

Advantages

- Fast and easy to implement on financial datasets
- Coefficients directly interpretable (e.g. impact of interest rate on return)
- Low computational cost is suitable for real-time prediction
- Good baseline model before testing complex models
- Works well when market relationships are approximately linear

Limits

- Financial markets are highly non-linear poor fit for complex patterns
- Very sensitive to outliers (market crashes, black swan events)
- Assumes no multicollinearity between financial indicators
- Cannot capture temporal dependencies or market memory
- Fails during high volatility periods (crisis, sudden shocks)

Use Cases

- Stock return forecasting by predicting next-day or weekly returns
- Portfolio optimization by estimating expected returns per asset
- Factor modeling by linking macroeconomic variables to asset returns
- Risk premium estimation by modeling relationship between risk and return
- Baseline benchmarking reference model to compare against LSTM or ANN

CLASSICAL ML : Random Forest

Advantages

- Handles non-linear relationships in financial data
- Robust to outliers and market noise
- Automatically ranks feature importance (e.g. which indicator matters most)
- Reduces overfitting compared to a single decision tree
- Works well with mixed financial features (technical + fundamental)

Limits

- Cannot capture temporal dependencies in price sequences
- Slow prediction time with large number of trees
- Less interpretable than Linear Regression
- Requires careful hyperparameter tuning (n_estimators, max_depth)
- Struggles with very high-frequency financial data

Use Cases

- Stock return classification : bullish vs bearish signal detection
- Credit risk modeling by predicting default probability
- Volatility forecasting by estimating market turbulence
- Algorithmic trading by generating buy/sell signals
- Fraud detection by identifying abnormal financial transactions
-

DEEP LEARNING : Artificial Neural Network

Advantages

- Captures complex non-linear relationships in financial markets
- Adapts to large volumes of financial data
- More powerful than Linear Regression for volatile markets
- Can combine multiple financial features simultaneously
- Flexible architecture easily customizable depth and width

Limits

- Requires large amounts of financial training data
- Risk of overfitting on noisy financial data
- Hard to explain predictions to regulator
- Computationally expensive to train
- Does not natively handle sequential/temporal structure of prices

Use Cases

- Return forecasting by predicting daily/weekly asset returns
- Market regime detection by identifying bull/bear market phases
- Options pricing by modeling complex derivative relationships
- Portfolio allocation by optimizing asset weight distribution
- Sentiment-driven prediction by combining news features with price data

DEEP LEARNING : Long Short-Term Memory

Advantages

- Natively handles sequential and temporal structure of financial data
- Captures long-term market dependencies and trends
- Remembers relevant past events (earnings, crises) over long sequences
- Outperforms traditional models on volatile time series
- Handles non-linear and dynamic market patterns effectively

Limits

- Very slow to train on long financial sequences
- Requires large historical datasets to generalize well
- Highly sensitive to hyperparameter choices (units, dropout, sequence length)
- Difficult to interpret for financial compliance
- Risk of overfitting on short or noisy financial time series

Use Cases

- Stock price forecasting by predicting next-day or multi-step returns
- Cryptocurrency prediction by modeling highly volatile digital asset returns
- Forex rate forecasting by predicting currency pair movements
- Anomaly detection by identifying unusual return patterns in sequences
- High-frequency trading by capturing micro-patterns in tick-by-tick data

EXPERIMENTAL SETUP

Experimental Setup

For infrastructure we used Google Colab with Apache PySpark for data processing and feature engineering,

and TensorFlow with Keras for the neural networks.

sklearn handled Linear Regression and Random Forest.

Pandas acted as the bridge between Spark and TensorFlow.

Infrastructure	
Platform:	Google Colab
Data Processing:	Apache PySpark 3.x
Deep Learning:	TensorFlow 2.x + Keras
Classical ML:	sklearn (LR, RF)
Bridge:	Pandas (Spark ↔ TensorFlow)
Scaling:	StandardScaler on all features

Evaluation Metrics

All four models were evaluated on exactly the same test set using RMSE and R^2

R^2 measures how well the model explains the data:

$R^2 = 1 \rightarrow$ perfect predictions
$R^2 = 0 \rightarrow$ no better than guessing the average
$R^2 < 0 \rightarrow$ worse than guessing the average

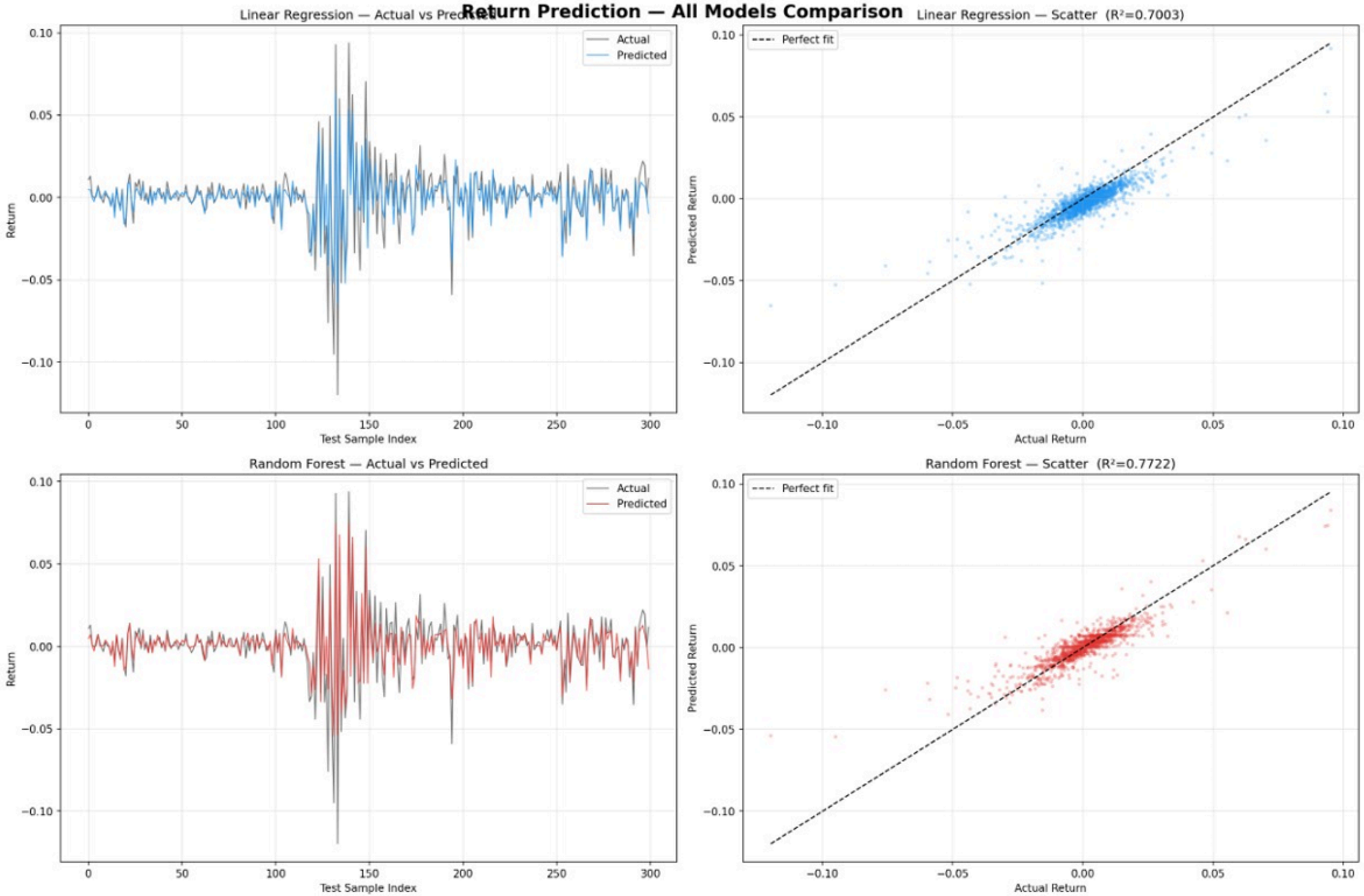
Evaluation Metrics	
RMSE	Root Mean Squared Error , average size of prediction error
R^2	Coefficient of determination 1 = perfect, 0 = no better than mean, <0 = worse than mean

Results : Model Performance

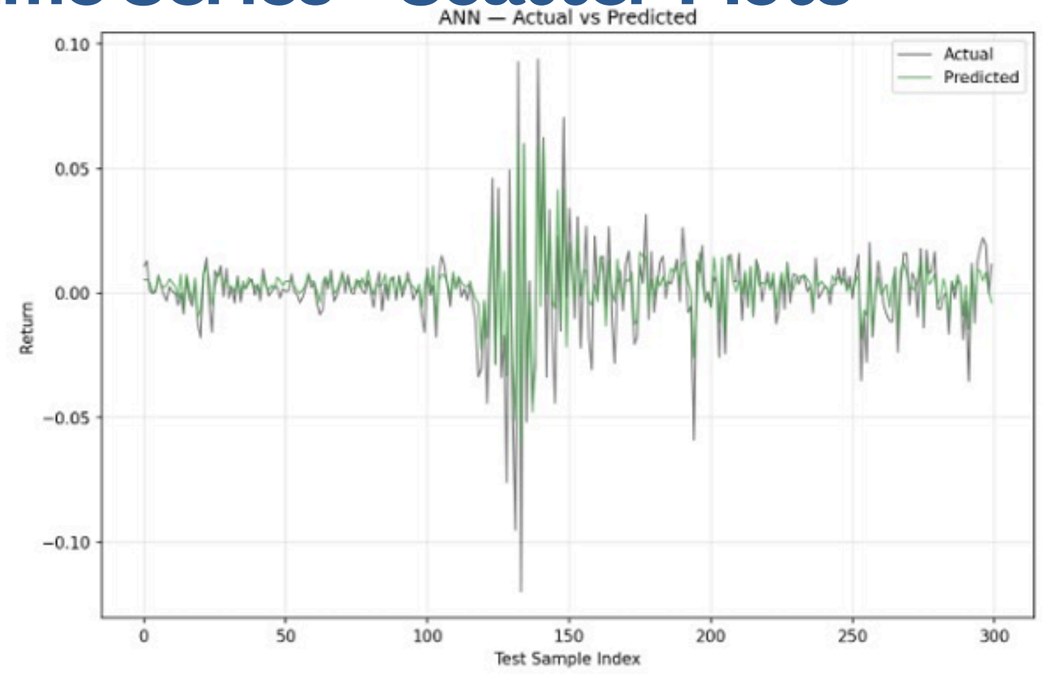
Model	R^2 (^GSPC)	RMSE	Rank	ΔR^2 vs LR
Linear Regression	0.7003	—	2nd	—
Random Forest	0.7722	Best	1st	+0.0719
ANN	0.6181	Competitive	3rd	−0.0822
LSTM	−0.0293	Worst	4th	−0.7296

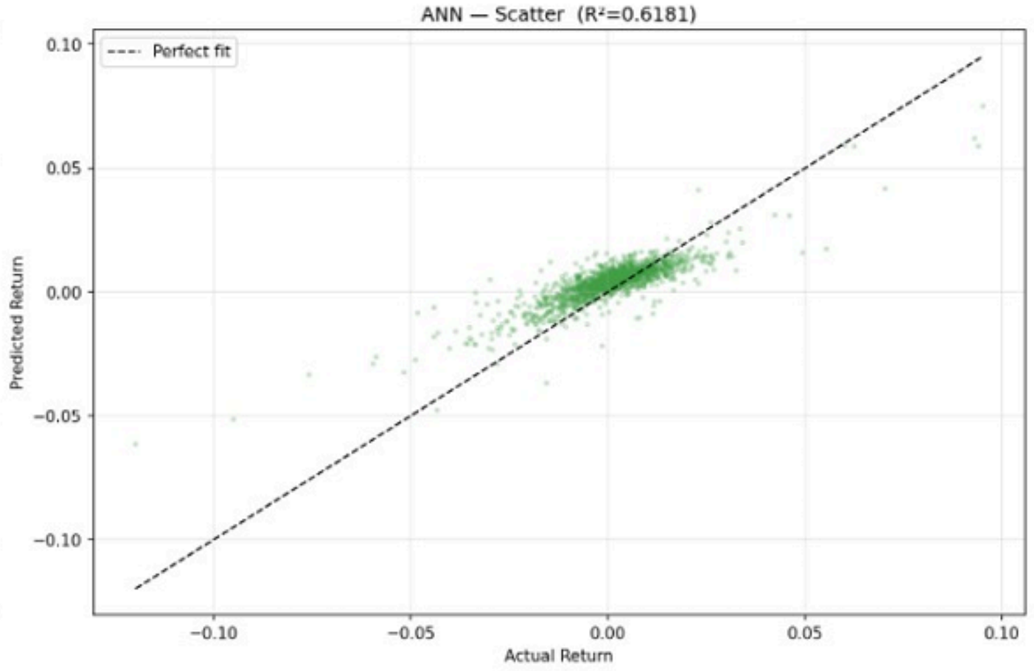
Best model : Random Forest

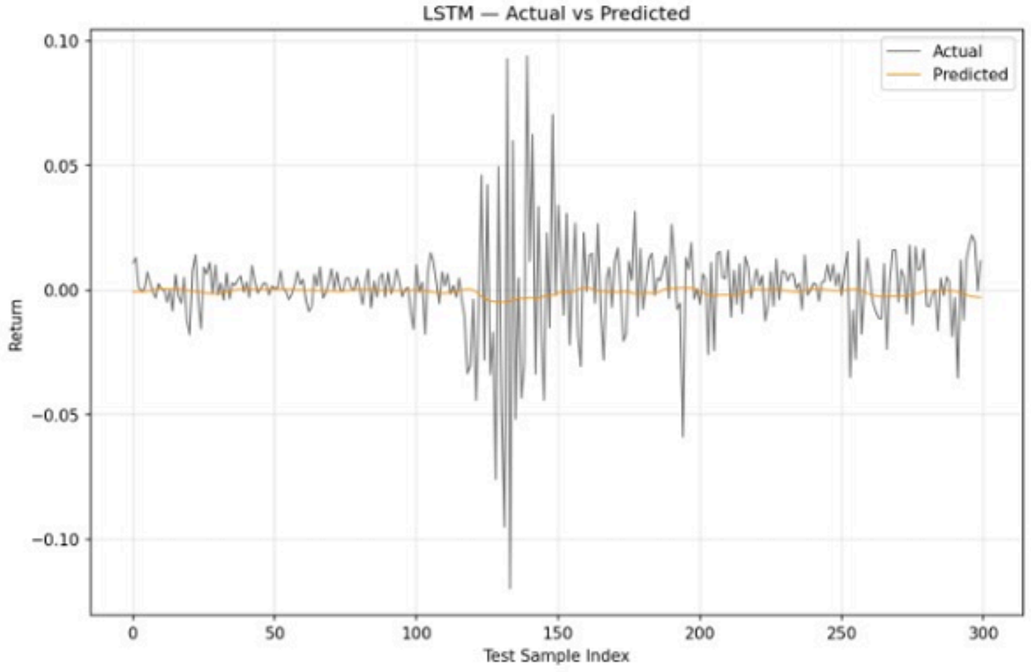
Figure 1 : All Models Time Series + Scatter Plots



All Models: Time Series + Scatter Plots







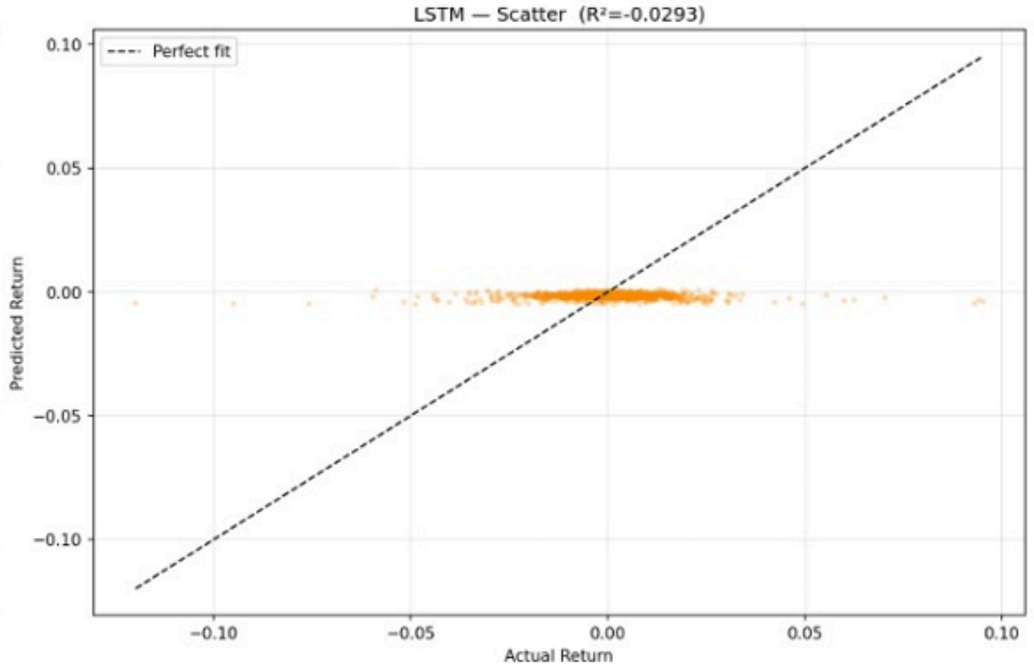


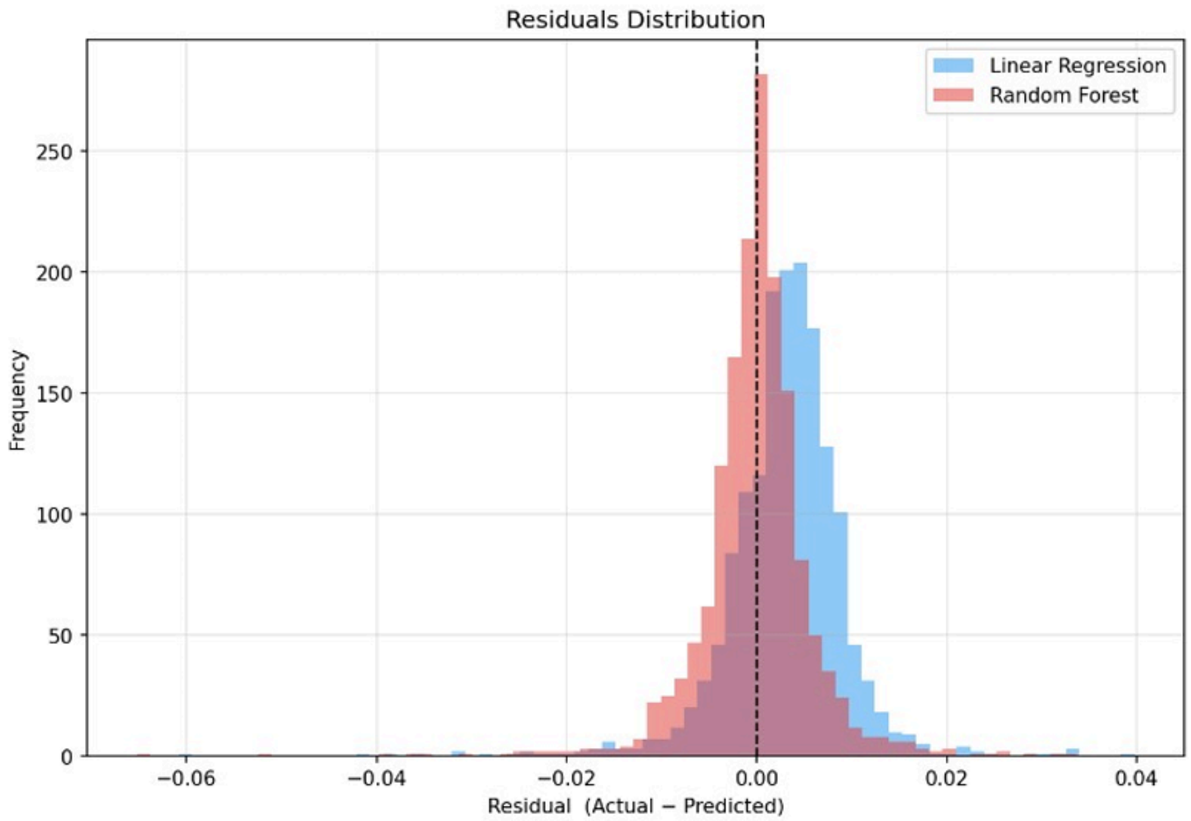
Figure 2 : Random Forest vs Linear Regression

Residuals

Both LR and RF errors are small and centered near zero which is good sign

Feature Imp.

Open_Close_gap dominates at ~75% ; the key intraday signal



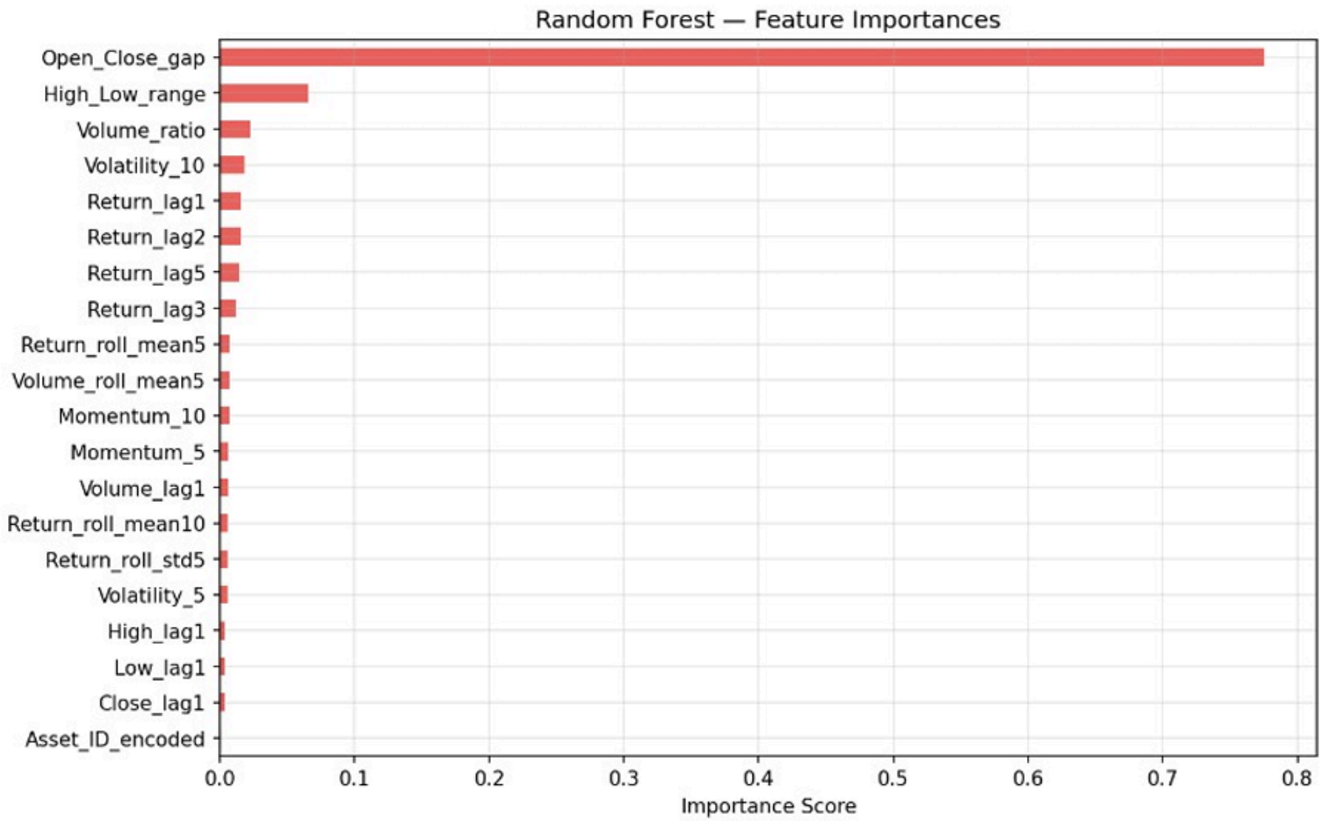
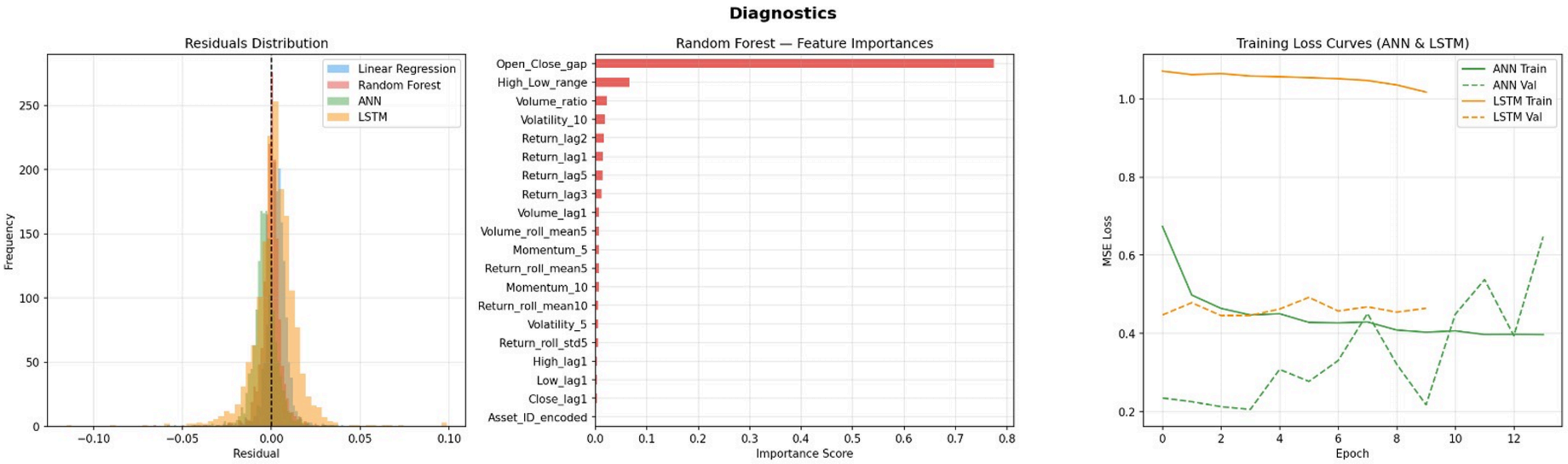


Figure 3 : Diagnostics Panel



Residuals Distribution

LR & RF: tight, centered at 0
LSTM: wide spread , random errors

RF Feature Importances

Open_Close_gap: ~75%
High_Low_range: ~10%
All others: <5% each

Training Loss Curves

ANN: converges smoothly
LSTM: val loss stays high & unstable

Libraries Used in the Project

Data manipulation

pandas

NumPy

Machine learning models

scikit learn

Deep learning models

TensorFlow

Data visualization

seaborn

matplotlib

Feature Engineering Pipeline

```python
def engineer_features(df):
    g = df.groupby('Asset_ID')
    df['Return'] = g['Close'].pct_change()
    # lag features so we don't leak future data into the model
    for lag in [1, 2, 3, 5]:
        df[f'Return_lag{lag}'] = g['Return'].shift(lag)
    df['Return_roll_mean5']  = g['Return'].transform(lambda x: x.shift(1).rolling(5).mean())
    df['Return_roll_std5']   = g['Return'].transform(lambda x: x.shift(1).rolling(5).std())
    df['Return_roll_mean10'] = g['Return'].transform(lambda x: x.shift(1).rolling(10).mean())
    df['Close_lag1']         = g['Close'].shift(1)
    df['High_Low_range']     = (df['High'] - df['Low']) / df['Close']
    df['Open_Close_gap']     = (df['Close'] - df['Open']) / df['Open']
    df['High_lag1']          = g['High'].shift(1)
    df['Low_lag1']           = g['Low'].shift(1)
    df['Volume_lag1']        = g['Volume'].shift(1)
    df['Volume_roll_mean5']  = g['Volume'].transform(lambda x: x.shift(1).rolling(5).mean())
    df['Volume_ratio']       = df['Volume_lag1'] / df['Volume_roll_mean5']
    df['Momentum_5']         = g['Close'].transform(lambda x: x.shift(1) / x.shift(6) - 1)
    df['Momentum_10']        = g['Close'].transform(lambda x: x.shift(1) / x.shift(11) - 1)
    df['Volatility_5']       = g['Return'].transform(lambda x: x.shift(1).rolling(5).std())
    df['Volatility_10']      = g['Return'].transform(lambda x: x.shift(1).rolling(10).std())
    return df
```

Transform raw financial data (price and volume) into meaningful features that help machine learning and deep learning models predict future returns.

Model Training : Classical ML

Linear Regression

```
print("\n  > Linear Regression")
lr_model = LinearRegression()
lr_model.fit(X_train_flat, y_train_flat)
y_pred_lr = lr_model.predict(X_test_flat)
```

- LinearRegression() → creates the model
- .fit() → trains it on historical data
- .predict() → generates return predictions on test data

Random Forest

```
print("  > Random Forest")
rf_model = RandomForestRegressor(
    n_estimators=300, max_depth=10,
    min_samples_leaf=5, random_state=42, n_jobs=-1)
rf_model.fit(X_train_flat, y_train_flat)
y_pred_rf = rf_model.predict(X_test_flat)
```

- RandomForestRegressor(...) → creates the model
- n_estimators=300 → builds 300 decision trees
- max_depth=10 → limits tree depth to avoid overfitting
- min_samples_leaf=5 → each leaf needs minimum 5 samples for stability
- random_state=42 → ensures reproducible results
- n_jobs=-1 → uses all CPU cores for faster training
- .fit() → trains all 300 trees
- .predict() → averages 300 tree predictions into one final output

Model Training : Deep Learning

ANN

```python
print("  > ANN (Artificial Neural Network)")
ann_model   = build_ann(n_features)
ann_history = ann_model.fit(
    X_train_flat, y_train_flat_sc,
    validation_split=0.1,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[early_stop],
    verbose=0
)
```

- build_ann(n_features) → builds the neural network architecture
- ann_model.fit() → trains the network
- X_train_flat, y_train_flat_sc → feeds scaled targets (required for neural nets)
- validation_split=0.1 → holds out 10% of training data to monitor overfitting
- epochs=EPOCHS → maximum number of training rounds
- batch_size=BATCH_SIZE → processes data in batches for efficiency
- callbacks=[early_stop] → automatically stops training when validation loss stops improving
- verbose=0 → runs silently without printing each epoch

LSTM

```python
print("  > LSTM (Long Short-Term Memory)")
lstm_model   = build_lstm(LSTM_TIMESTEPS, n_features)
lstm_history = lstm_model.fit(
    X_train_seq, y_train_seq,
    validation_split=0.1,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[early_stop],
    verbose=0
)
```

- build_lstm(LSTM_TIMESTEPS, n_features) → builds the LSTM architecture with sequence input
- lstm_model.fit() → trains the network
- X_train_seq, y_train_seq → feeds 3D sequential data (samples, timesteps, features) instead of flat data
- validation_split=0.1 → same as ANN
- epochs=EPOCHS → same as ANN
- callbacks=[early_stop] → same early stopping as ANN
- verbose=0 → silent training

LSTM UNDERPERFORMANCE ANALYSIS

R^2

-0.0293

Worse than guessing mean

~1,262	Training sequences per asset ; far below what LSTM needs to learn gate parameters
10 days	Sequence window too short to capture meaningful long-range patterns

Dataset was too small for deep sequence learning.

What LSTM Actually Needs to Succeed

- 10,000+ training sequences (we had ~5,000 total)
- More assets or longer history to generate sequences
- Lighter architecture : fewer parameters for this data size

KEY FINDING : FEATURE IMPORTANCE

Open–Close gap and High–Low range drove most of the predictive power across all models

01

Random Forest wins

R^2=0.7722 , 100 shallow trees averaging predictions beat all neural networks. Stable, non-linear, hard to overfit on small datasets.

02

Linear Regression

R^2=0.7003 , a weighted sum of features nearly matched the best model. Suggests most of the signal is actually linear.

03

ANN is decent

R^2=0.6181 , the network learned but couldn't extract more than the simpler models. Small dataset limited its capacity.

04

LSTM completely fails

R^2=−0.0293 — only ~1,262 sequences per asset. Far too few for LSTM gates to learn temporal patterns. Predicted near zero for everything.

05

Intraday features dominate

Open-Close gap and High-Low range drove most of the predictive power across ALL models. More informative than lagged returns.